

计算物理学作业

1900011604 张植竣 北京大学

2021 年 11 月 13 日

1.(a) 使用 Neville 算法构造内插多项式:

```

1  import numpy as np
2  import matplotlib
3  import matplotlib.pyplot as plt
4
5  matplotlib.rcParams["text.usetex"] = True
6
7
8  def times(a_array, b):
9      n = len(a_array)
10     x_array = np.zeros(n)
11     x_array[0] = 0 - a_array[0] * b
12     for i in range(1, n - 1):
13         x_array[i] = a_array[i - 1] - a_array[i] * b
14     x_array[n - 1] = a_array[-2]
15     return x_array
16
17
18 def Neville(x, y, x_array):
19     n = len(x)
20     Nev_array = np.zeros((n, n, n))
21     for j in range(n):
22         Nev_array[j, 0, 0] = y[j]
23     for j in range(1, n):
24         for k in range(1, j + 1):
25             A = times(Nev_array[j, k - 1], x[j - k])
26             B = times(Nev_array[j - 1, k - 1], x[j])
27             Nev_array[j][k] = (A - B) / (x[j] - x[j - k])
28     ans = Nev_array[-1][-1]
29     print(ans)
30     y_array = np.zeros(len(x_array))
31     for i in range(len(ans)):
32         y_array += ans[i] * x_array ** i
33     return y_array
34
35
36 # 1.(a)
37
38 x = np.linspace(0, np.pi / 2, num=5, endpoint=True)
39 y = np.sin(x)
40
41 x_array = np.linspace(min(x), max(x), num=10001, endpoint=True)
42 y_array = Neville(x, y, x_array)
43
44 y_ref = np.sin(x_array)
45 delta_max = max(abs(y_ref - y_array))

```

```

46 print("delta_max =", delta_max)
47
48 fig, ax = plt.subplots(figsize=(16, 10), dpi=300)
49 ax.set_xlabel(r'$x$')
50 ax.set_ylabel(r'$y$')
51 ax.scatter(x, y, color="r", marker="o")
52 ax.plot(x_array, y_array, color="b", label="Neville Interpolation")
53 ax.legend(loc="upper left")
54 plt.show()

```

给出内插多项式为:

$$P_4(x) = 0.99631698x + 0.01995143x^2 - 0.20358546x^3 + 0.02871423x^4$$

内插效果如图:

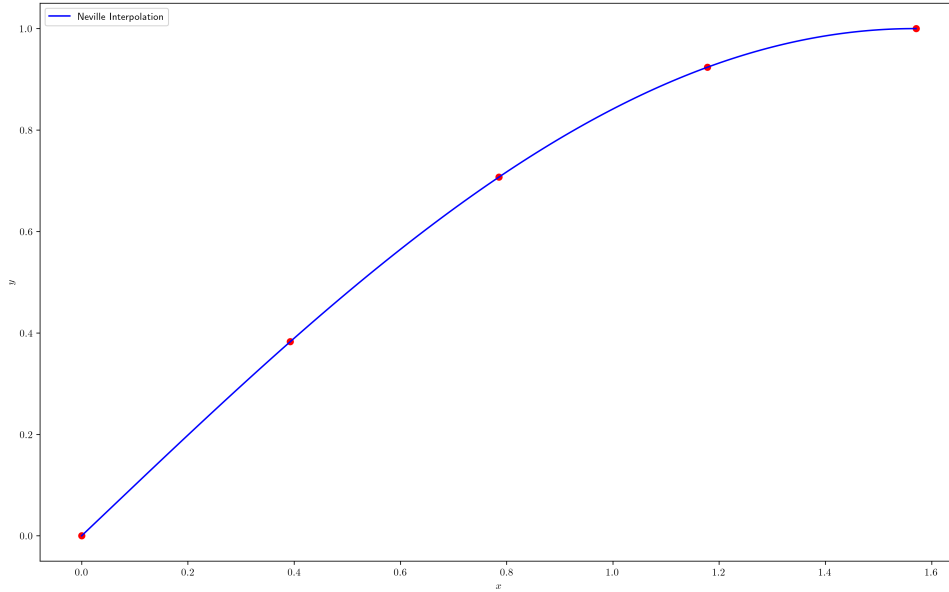


图 1: 使用 Neville 算法进行多项式内插

通过计算等距的 10001 个节点 ξ 的误差 $\delta = |f(\xi) - P_n(\xi)|$, 估计误差最大为:

$$\delta_{max} = 0.00021533$$

而估计任意节点选择下, 最大可能的误差为:

$$\mathcal{A} = \max |f(x) - P_n(x)| \leq \frac{|x_n - x_0|^n}{(n+1)!} \max_{x_0 \leq \xi \leq x_n} |f^{(n+1)}(\xi)|$$

即:

$$\mathcal{A}_{max} = \frac{|\frac{\pi}{2} - 0|^4}{(4+1)!} \max_{0 \leq \xi \leq 1} \left| \frac{d^{(4+1)}}{dx^{(4+1)}} \sin(\xi) \right| = 0.05073390$$

可见有 $\delta_{max} < \mathcal{A}_{max}$

1.(b) 选择节点为:

$$x_0 = 0, x_1 = 0.1, x_2 = 0.2, x_3 = 0.3, x_4 = \frac{\pi}{2}$$

给出内插多项式为:

$$P_4(x) = 0.99992807x + 0.00136841x^2 - 0.17477156x^3 + 0.01697043x^4$$

通过计算等距的 10001 个节点 ξ 的误差 $\delta = |f(\xi) - P_n(\xi)|$, 估计误差最大为:

$$\delta_{max} = 0.00316623 < \Delta_{max}$$

仍小于各种可能的误差最大值。

1.(c) 由题意, 需要找到最小的 n 使其满足

$$\Delta_{max}(n) = \frac{1}{(n+1)!} \left(\frac{\pi}{2}\right)^n < 10^{-8}$$

因为: ($\Delta_{max}(n)$ 显然是一个单调递减数列)

$$\Delta_{max}(12) = 3.62374983 \times 10^{-8} > 10^{-8} > \Delta_{max}(13) = 4.06583780 \times 10^{-9}$$

故满足条件的最小 n 值为 13, 即至少需要 14 个节点 (包括头尾) 进行插值。

2.(a) 同样使用 Neville 算法, 构造内插多项式为:

$$\begin{aligned}
 P_{20}(x) = & 1 + 7.21644966 \times 10^{-16}x - 2.41434796 \times 10^1x^2 - 6.21724894 \times 10^{-14}x^3 + 4.70846227 \times 10^2x^4 \\
 & + 8.81072992 \times 10^{-13}x^5 - 6.11921995 \times 10^3x^6 - 9.77706804 \times 10^{-12}x^7 + 4.93175429 \times 10^4x^8 \\
 & + 5.09317033 \times 10^{-11}x^9 - 2.45249284 \times 10^5x^{10} - 2.03726813 \times 10^{-10}x^{11} + 7.57287093 \times 10^5x^{12} \\
 & + 6.69388101 \times 10^{-10}x^{13} - 1.44294496 \times 10^6x^{14} - 8.73114914 \times 10^{-10}x^{15} + 1.63917741 \times 10^6x^{16} \\
 & + 6.98491931 \times 10^{-10}x^{17} - 1.01209487 \times 10^6x^{18} - 3.05590220 \times 10^{-10}x^{19} + 2.60178631 \times 10^5x^{20}
 \end{aligned}$$

内插效果如图:

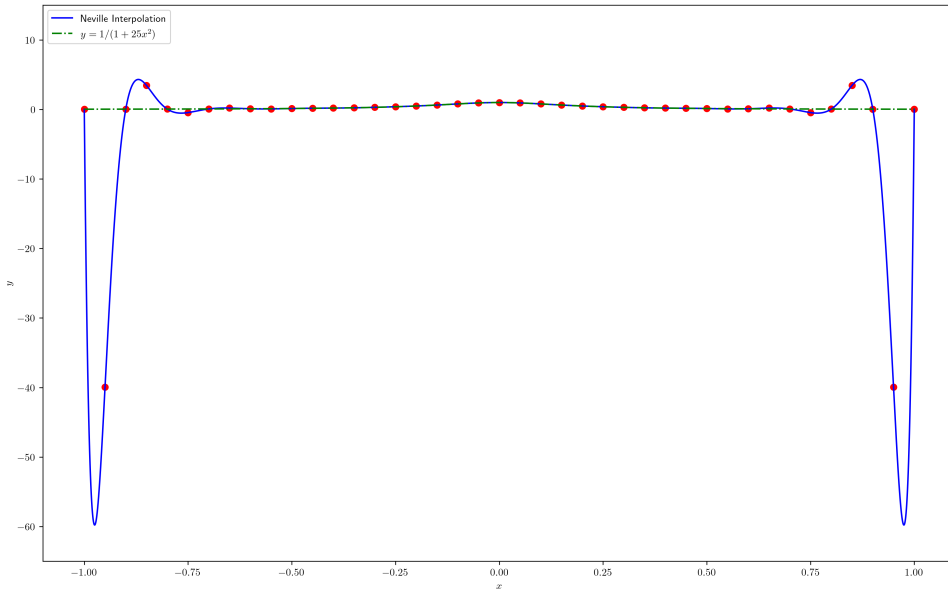


图 2: 使用 Neville 算法进行多项式内插, 探究 Runge 现象

每个节点和区间中点的 x , $f(x)$, $P_{20}(x)$ 以及 $|f(x) - P_{20}(x)|$ 列表如下:

x	$f(x)$	$P_{20}(x)$	$ f(x) - P_{20}(x) $	x	$f(x)$	$P_{20}(x)$	$ f(x) - P_{20}(x) $
-1.00	0.0384615	0.0384615	1.38803×10^{-10}	-0.95	0.0424403	-39.9524	39.9949
-0.90	0.0470588	0.0470588	9.09923×10^{-11}	-0.85	0.052459	3.45496	3.4025
-0.80	0.0588235	0.0588235	2.27909×10^{-11}	-0.75	0.06639	-0.447052	0.513442
-0.70	0.0754717	0.0754717	4.94807×10^{-12}	-0.65	0.0864865	0.202423	0.115936
-0.60	0.1	0.1	4.97741×10^{-13}	-0.55	0.116788	0.08066	0.0361283
-0.50	0.137931	0.137931	4.84335×10^{-14}	-0.45	0.164948	0.179763	0.0148142
-0.40	0.2	0.2	3.69149×10^{-15}	-0.35	0.246154	0.238446	0.00770791
-0.30	0.307692	0.307692	2.22045×10^{-16}	-0.25	0.390244	0.395093	0.00484915
-0.20	0.5	0.5	3.33067×10^{-16}	-0.15	0.64	0.636755	0.00324466
-0.10	0.8	0.8	1.11022×10^{-16}	-0.05	0.941176	0.94249	0.00131391
0.00	1	1	0	0.05	0.941176	0.94249	0.00131391

x	$f(x)$	$P_{20}(x)$	$ f(x) - P_{20}(x) $	x	$f(x)$	$P_{20}(x)$	$ f(x) - P_{20}(x) $
0.10	0.8	0.8	0	0.15	0.64	0.636755	0.00324466
0.20	0.5	0.5	1.66533×10^{-16}	0.25	0.390244	0.395093	0.00484915
0.30	0.307692	0.307692	1.99840×10^{-15}	0.35	0.246154	0.238446	0.00770791
0.40	0.2	0.2	3.63598×10^{-15}	0.45	0.164948	0.179763	0.0148142
0.50	0.137931	0.137931	2.47302×10^{-14}	0.55	0.116788	0.08066	0.0361283
0.60	0.1	0.1	2.13496×10^{-13}	0.65	0.0864865	0.202423	0.115936
0.70	0.0754717	0.0754717	4.52111×10^{-13}	0.75	0.06639	-0.447052	0.513442
0.80	0.0588235	0.0588235	1.00579×10^{-11}	0.85	0.052459	3.45496	3.4025
0.90	0.0470588	0.0470588	1.09567×10^{-11}	0.95	0.0424403	-39.9524	39.9949
1.00	0.0384615	0.0384615	2.23876×10^{-11}	-	-	-	-

可见在 $0.7 < |x| < 1$ 的范围内，误差 $|f(x) - P_{20}(x)|$ 急剧增大，最大误差为：

$$\delta_{max} = 39.9948893515$$

这甚至远远超过了 $f(x)$ 的最大值 $f_{max} = 1$ 。

2.(b) 取阶数 $N = 20$ 的标准 Chebyshev 节点如下：

$$x_k = \cos\left(\frac{\pi(k+1/2)}{20}\right), \quad k = 0, 1, \dots, 19$$

分别用公式：

$$c_m = \frac{2}{N} \sum_{k=0}^{N-1} f\left[\cos\left(\frac{\pi(k+1/2)}{20}\right)\right] \cos\left(\frac{\pi m(k+1/2)}{20}\right)$$

以及 Clenshaw 迭代算法，构造目标函数的近似多项式：

$$f(x) \approx \frac{c_0}{2} + \sum_{k=1}^{N-1} c_k T_k(x) = \sum_{k=1}^N s_k x^k$$

代码如下：

```

1 import numpy as np
2 import matplotlib
3 import matplotlib.pyplot as plt
4
5 matplotlib.rcParams["text.usetex"] = True
6
7
8 def func(x):
9     return 1 / (1 + 25 * x ** 2)
10
11
12 n = 20
13 x = np.array([np.cos(np.pi * (m + 0.5) / n) for m in range(n)])
14 y = func(x)
15

```

```

16 # Chebyshev
17
18
19 def Chebyshev(func, n, x_array):
20     x = np.array([np.cos(np.pi * (m + 0.5) / n) for m in range(n)])
21     y = func(x)
22     c = np.zeros(n)
23     for k in range(n):
24         for m in range(n):
25             c[k] += (2 / n) * y[m] * np.cos(np.pi * k * (m + 0.5) / n)
26     print(c)
27     y_array = np.zeros(len(x_array))
28     for i in range(1, len(c)):
29         y_array += c[i] * np.cos(i * np.arccos(x_array))
30     y_array += 1 / 2 * c[0]
31     return y_array
32
33
34 x_array = [np.cos(np.pi * (m + 0.5) / n) for m in range(n)]
35 for i in range(n - 1):
36     x_array.append((x[i] + x[i + 1]) / 2)
37 x_array = np.array(sorted(x_array))
38 y_array = Chebyshev(func, n, x_array)
39
40 y_ref = func(x_array)
41 delta = abs(y_ref - y_array)
42 delta_max = max(delta)
43
44 x_plot = np.linspace(min(x), max(x), num=10001, endpoint=True)
45 y_plot = Chebyshev(func, n, x_plot)
46
47 y_ref_plot = func(x_plot)
48 delta_max = max(abs(y_ref - y_array))
49
50 Array = np.array([x_array, y_ref, y_array, delta]).T
51 print("delta_max =", delta_max)
52 print(Array)
53
54 fig, ax = plt.subplots(figsize=(16, 10), dpi=300)
55 ax.set_xlabel(r'$x$')
56 ax.set_ylabel(r'$y$')
57 ax.scatter(x_array, y_array, color="r", marker="o")
58 ax.plot(x_plot, y_plot, color="b", label="Chebyshev Interpolation")
59 ax.plot(x_plot, y_ref_plot, color="g", linestyle="-. ", label=r"$y=1/(1+25x^2)$")
60 ax.legend(loc="upper left")
61 plt.show()
62
63

```

```

64 # Chebyshev-Clenshaw
65
66
67 def Cheb_Clen(func, n, x_array):
68     x = np.array([np.cos(np.pi * (m + 0.5) / n) for m in range(n)])
69     y = func(x)
70     c = np.zeros(n + 1)
71     for k in range(n):
72         for m in range(n):
73             c[k] += (2 / n) * y[m] * np.cos(np.pi * k * (m + 0.5) / n)
74     b = np.zeros((n + 2, n))
75     for k in range(2, n + 2):
76         b[k] = np.append(c[-k + 1], 2 * b[k - 1][: -1]) - b[k - 2]
77     S = np.append(0.5 * c[0], b[-1][: -1]) - b[-2]
78     print(S)
79     y_array = np.zeros(len(x_array))
80     for i in range(len(S)):
81         y_array += S[i] * x_array ** i
82     return y_array
83
84
85 x_array = [np.cos(np.pi * (m + 0.5) / n) for m in range(n)]
86 for i in range(n - 1):
87     x_array.append((x[i] + x[i + 1]) / 2)
88 x_array = np.array(sorted(x_array))
89 y_array = Cheb_Clen(func, n, x_array)
90
91 y_ref = func(x_array)
92 delta = abs(y_ref - y_array)
93 delta_max = max(delta)
94
95 x_plot = np.linspace(min(x), max(x), num=10001, endpoint=True)
96 y_plot = Cheb_Clen(func, n, x_plot)
97
98 y_ref_plot = func(x_plot)
99 delta_max = max(abs(y_ref - y_array))
100
101 Array = np.array([x_array, y_ref, y_array, delta]).T
102 print("delta_max =", delta_max)
103 print(Array)
104
105 fig, ax = plt.subplots(figsize=(16, 10), dpi=300)
106 ax.set_xlabel(r'$x$')
107 ax.set_ylabel(r'$y$')
108 ax.scatter(x_array, y_array, color="r", marker="o")
109 ax.plot(x_plot, y_plot, color="b", label="Chebyshev-Clenshaw Interpolation")
110 ax.plot(x_plot, y_ref_plot, color="g", linestyle="-. ", label=r"$y=1/(1+25x^2)$")
111 ax.legend(loc="upper left")

```


系数矩阵分别为:

$$C = \begin{bmatrix} 3.91955054 \times 10^{-1} & 9.97465999 \times 10^{-18} & -2.63311458 \times 10^{-1} & 1.12757026 \times 10^{-17} \\ 1.76797696 \times 10^{-1} & -1.16660154 \times 10^{-16} & -1.18571565 \times 10^{-1} & 1.55691432 \times 10^{-16} \\ 7.93168838 \times 10^{-2} & 1.56125113 \times 10^{-17} & -5.27529045 \times 10^{-2} & -5.37764278 \times 10^{-17} \\ 3.46293898 \times 10^{-2} & -4.46691295 \times 10^{-17} & -2.20465775 \times 10^{-2} & 2.15756232 \times 10^{-16} \\ 1.29912177 \times 10^{-2} & -3.81639165 \times 10^{-17} & -6.01445262 \times 10^{-3} & -2.86229374 \times 10^{-17} \end{bmatrix}$$

$$S = \begin{bmatrix} 9.62409671 \times 10^{-1} & -4.88693287 \times 10^{-15} & -1.65421760 \times 10^1 & 1.32987371 \times 10^{-13} \\ 1.65458229 \times 10^2 & -1.24297100 \times 10^{-12} & -9.60824747 \times 10^2 & 4.65544270 \times 10^{-12} \\ 3.37901728 \times 10^3 & -5.00327557 \times 10^{-12} & -7.41345341 \times 10^3 & -1.29798394 \times 10^{-11} \\ 1.01954749 \times 10^4 & 4.59232652 \times 10^{-11} & -8.53489389 \times 10^3 & -5.71169778 \times 10^{-11} \\ 3.97316472 \times 10^3 & 3.31397132 \times 10^{-11} & -7.88326334 \times 10^2 & -7.50333129 \times 10^{-12} \end{bmatrix}$$

内插效果如图:

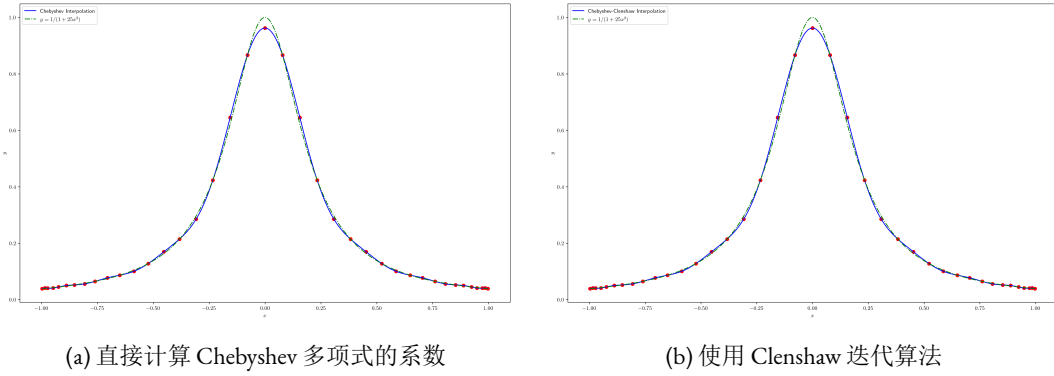


图 3: 使用 Chebyshev 多项式进行内插

两种计算方法给出的内插多项式在精度允许范围内一致, 通过计算等距的 10001 个节点 ξ 的误差 $\delta = |f(\xi) - C_n(\xi)|$ 估计最大误差均为:

$$\delta_{max} = 0.0375903289$$

每个节点和区间中点的 x , $f(x)$, $C_{20}(x)$ 以及 $|f(x) - C_{20}(x)|$ 列表如下:

x	$f(x)$	$C_{20}(x)$	$ f(x) - C_{20}(x) $
-9.96917×10^{-1}	3.86906×10^{-2}	3.86906×10^{-2}	1.73953×10^{-12}
-9.84644×10^{-1}	3.96227×10^{-2}	4.10124×10^{-2}	1.38975×10^{-3}
-9.72370×10^{-1}	4.05884×10^{-2}	4.05884×10^{-2}	9.80646×10^{-13}
-9.48125×10^{-1}	4.26012×10^{-2}	4.10278×10^{-2}	1.57346×10^{-3}
-9.23880×10^{-1}	4.47651×10^{-2}	4.47651×10^{-2}	2.25382×10^{-3}

x	$f(x)$	$C_{20}(x)$	$ f(x) - C_{20}(x) $
-8.88260×10^{-1}	4.82506×10^{-2}	5.00512×10^{-2}	1.80065×10^{-3}
-8.52640×10^{-1}	5.21516×10^{-2}	5.21516×10^{-2}	1.78940×10^{-13}
-8.06523×10^{-1}	5.79308×10^{-2}	5.57609×10^{-2}	2.16985×10^{-3}
-7.60406×10^{-1}	6.47022×10^{-2}	6.47022×10^{-2}	1.11189×10^{-13}
-7.04927×10^{-1}	7.44987×10^{-2}	7.72938×10^{-2}	2.79513×10^{-3}
-6.49448×10^{-1}	8.66208×10^{-2}	8.66208×10^{-2}	3.45557×10^{-15}
-5.85973×10^{-1}	1.04339×10^{-1}	1.00421×10^{-1}	3.91822×10^{-3}
-5.22499×10^{-1}	1.27794×10^{-1}	1.27794×10^{-1}	2.16493×10^{-15}
-4.52591×10^{-1}	1.63373×10^{-1}	1.69511×10^{-1}	6.13821×10^{-3}
-3.82683×10^{-1}	2.14539×10^{-1}	2.14539×10^{-1}	1.94289×10^{-16}
-3.08064×10^{-1}	2.96508×10^{-1}	2.85364×10^{-1}	1.11436×10^{-2}
-2.33445×10^{-1}	4.23295×10^{-1}	4.23295×10^{-1}	7.77156×10^{-16}
-1.55952×10^{-1}	6.21880×10^{-1}	6.45255×10^{-1}	2.33756×10^{-2}
-7.84591×10^{-2}	8.66629×10^{-1}	8.66629×10^{-1}	1.11022×10^{-16}
6.24500×10^{-7}	$1.00000 \times 10^{+0}$	9.62410×10^{-1}	3.75903×10^{-2}
7.84591×10^{-2}	8.66629×10^{-1}	8.66629×10^{-1}	4.44089×10^{-16}
1.55952×10^{-1}	6.21880×10^{-1}	6.45255×10^{-1}	2.33756×10^{-2}
2.33445×10^{-1}	4.23295×10^{-1}	4.23295×10^{-1}	5.55112×10^{-16}
3.08064×10^{-1}	2.96508×10^{-1}	2.85364×10^{-1}	1.11436×10^{-2}
3.82683×10^{-1}	2.14539×10^{-1}	2.14539×10^{-1}	2.77556×10^{-16}
4.52591×10^{-1}	1.63373×10^{-1}	1.69511×10^{-1}	6.13821×10^{-3}
5.22499×10^{-1}	1.27794×10^{-1}	1.27794×10^{-1}	1.36002×10^{-15}
5.85973×10^{-1}	1.04339×10^{-1}	1.00421×10^{-1}	3.91822×10^{-3}
6.49448×10^{-1}	8.66208×10^{-2}	8.66208×10^{-2}	6.89726×10^{-15}
7.04927×10^{-1}	7.44987×10^{-2}	7.72938×10^{-2}	2.79513×10^{-3}
7.60406×10^{-1}	6.47022×10^{-2}	6.47022×10^{-2}	7.46070×10^{-14}
8.06523×10^{-1}	5.79308×10^{-2}	5.57609×10^{-2}	2.16985×10^{-3}
8.52640×10^{-1}	5.21516×10^{-2}	5.21516×10^{-2}	2.36894×10^{-14}
8.88260×10^{-1}	4.82506×10^{-2}	5.00512×10^{-2}	1.80065×10^{-3}
9.23880×10^{-1}	4.47651×10^{-2}	4.47651×10^{-2}	1.52843×10^{-13}
9.48125×10^{-1}	4.26012×10^{-2}	4.10278×10^{-2}	1.57346×10^{-3}
9.72370×10^{-1}	4.05884×10^{-2}	4.05884×10^{-2}	1.83474×10^{-12}
9.84644×10^{-1}	3.96227×10^{-2}	4.10124×10^{-2}	1.38975×10^{-3}
9.96917×10^{-1}	3.86906×10^{-2}	3.86906×10^{-2}	2.02421×10^{-13}

与 2.(a) 中的 Neville 算法内插比较, 可见 x 在靠近边界处的误差明显减小, 不会出现极端的误差情况。Chebyshev 算法的最大误差出现在 $x = 0$ 的点, 说明 Chebyshev 多项式的值域限制 $[-1, 1]$ 使得其对变化剧烈的函数内插精度不高。

2.(c) 使用 21 个等距节点构造三次样条函数，边界条件为 Neumann 边界条件：

$$f'(-1) = S'_d(Y; -1) = -\frac{25}{338}, \quad f'(1) = S'_d(Y; 1) = \frac{25}{338}$$

代码如下：

```

1  import numpy as np
2  import matplotlib
3  import matplotlib.pyplot as plt
4
5  matplotlib.rcParams["text.usetex"] = True
6
7
8  def Thomas_LU(matrix, vec):
9      n = len(matrix)
10     L = np.zeros((n, n))
11     U = np.zeros((n, n))
12     U[0, 0] = matrix[0, 0]
13     for i in range(n):
14         L[i, i] = 1
15     for i in range(1, n):
16         L[i, i - 1] = matrix[i, i - 1] / U[i - 1, i - 1]
17         U[i, i] = matrix[i, i] - L[i, i - 1] * matrix[i - 1, i]
18         U[i - 1, i] = matrix[i - 1, i]
19     # L y = vec
20     L_vec = np.column_stack((L, vec))
21     y = np.zeros(n)
22     # 因为 L 是下三角矩阵，这里的反代顺序与通常的相反
23     y[0] = L_vec[0][n] / L_vec[0][0]
24     for i in range(1, n):
25         Sum3 = 0
26         for k in range(0, i):
27             Sum3 += L_vec[i][k] * y[k]
28         y[i] = (L_vec[i][n] - Sum3) / L_vec[i][i]
29     # U x = y
30     U_y = np.column_stack((U, y))
31     x = np.zeros(n)
32     x[n - 1] = U_y[n - 1][n] / U_y[n - 1][n - 1]
33     for i in range(n - 2, -1, -1):
34         Sum4 = 0
35         for k in range(i, n):
36             Sum4 += U_y[i][k] * x[k]
37         x[i] = (U_y[i][n] - Sum4) / U_y[i][i]
38     return x
39
40
41 def Spline(x, y, edge_type, x_array, *edge):
42     n = len(x) - 1 # 这保证了 x 编号为 0 ~ n

```

```

43 matrix = np.zeros((n + 1, n + 1)) # n+1 阶矩阵, 与 x 的个数相同
44 vec = np.zeros(n + 1) # n+1 维向量, 与 x 的个数相同
45 h = np.zeros(n) # n 维向量, 编号为 0 ~ n-1, h[j] = x[j+1]-x[j]
46 for j in range(n):
47     h[j] = x[j + 1] - x[j]
48 for j in range(1, n):
49     matrix[j, j - 1] = h[j - 1] / 6
50     matrix[j, j] = (h[j - 1] + h[j]) / 3
51     matrix[j, j + 1] = h[j] / 6
52     vec[j] = (y[j + 1] - y[j]) / h[j] - (y[j] - y[j - 1]) / h[j - 1]
53 if edge_type == 1: # 边界条件为: 边界处二阶导的值为0
54     matrix[0, 0] = 1
55     matrix[0, 1] = 0
56     matrix[n, n - 1] = 0
57     matrix[n, n] = 1
58     vec[0] = 0
59     vec[n] = 0
60     M = Thomas_LU(matrix, vec)
61 elif edge_type == 2: # 边界条件为: x_0 处导数值为 edge_0, x_n 处导数值为 edge_n
62     if len(edge) == 2:
63         edge_0 = edge[0]
64         edge_n = edge[1]
65     else:
66         print("Type 2 (Neumann) edge error")
67         return
68     matrix[0, 0] = -h[0] / 3
69     matrix[0, 1] = -h[0] / 6
70     matrix[n, n - 1] = h[n - 1] / 6
71     matrix[n, n] = h[n - 1] / 3
72     vec[0] = edge_0 - (y[1] - y[0]) / x[1]
73     vec[n] = edge_n - (y[n] - y[n - 1]) / x[n]
74     M = Thomas_LU(matrix, vec)
75 elif edge_type == 3: # 周期性边界条件
76     print(matrix)
77     print(vec)
78     matrix = matrix[:-1, :-1] # 去掉最后一行, 变成 n 阶矩阵, 编号为 0 ~ n-1
79     vec = vec[:-1]
80     matrix[0, 0] = (h[0] + h[n - 1]) / 3
81     matrix[0, 1] = h[0] / 6
82     matrix[0, n - 1] = h[n - 1] / 6
83     matrix[n - 1, 0] = h[n - 1] / 6
84     vec[0] = (y[1] - y[0]) / x[1] - (y[n] - y[n - 1]) / x[n]
85     b = np.zeros(n - 1)
86     b[0] = -matrix[1, 0]
87     b[n - 2] = -matrix[n - 1, 0]
88     u = Thomas_LU(matrix[1:, 1:], vec[1:])
89     v = Thomas_LU(matrix[1:, 1:], b)
90     M = np.zeros(n + 1) # M0 = Mn, 故只需要解n个值

```

```

91     M0_u = vec[0] - matrix[0, 1] * u[1] - matrix[0, n - 1] * u[n - 2]
92     M0_v = matrix[0, 0] + matrix[0, 1] * v[1] + matrix[0, n - 1] * v[n - 2]
93     M[0] = M0_u / M0_v
94     for i in range(1, n):
95         M[i] = u[i - 1] + M[0] * v[i - 1]
96     M[n] = M[0]
97     S = np.zeros((n, 4))
98     for j in range(n):
99         S[j, 3] = (M[j + 1] - M[j]) / (6 * h[j])
100        S[j, 2] = (M[j] * x[j + 1] - M[j + 1] * x[j]) / (2 * h[j])
101        A_j = (y[j + 1] - y[j]) / h[j] - h[j] * (M[j + 1] - M[j]) / 6
102        S[j, 1] = (M[j + 1] * x[j] ** 2 - M[j] * x[j + 1] ** 2) / (2 * h[j]) + A_j
103        B_j = y[j] - (M[j] * h[j] ** 2) / 6
104        S[j, 0] = (
105            (M[j] * x[j + 1] ** 3 - M[j + 1] * x[j] ** 3) / (6 * h[j])
106            - A_j * x[j]
107            + B_j
108        )
109        y_array = np.zeros(len(x_array))
110        y_array[0] = y[0]
111        for i in range(len(x_array)):
112            for j in range(n):
113                if x[j] < x_array[i] and x_array[i] <= x[j + 1]:
114                    y_array[i] += (
115                        S[j, 0]
116                        + S[j, 1] * x_array[i]
117                        + S[j, 2] * x_array[i] ** 2
118                        + S[j, 3] * x_array[i] ** 3
119                    )
120        return y_array
121
122
123 x = np.linspace(-1, 1, num=21, endpoint=True)
124 y = 1 / (1 + 25 * x ** 2)
125
126 edge_type = 2
127 edge_0 = -25 / 338
128 edge_n = 25 / 338
129
130 x_array = np.linspace(min(x), max(x), num=41, endpoint=True)
131 y_array = Spline(x, y, edge_type, x_array, edge_0, edge_n)
132
133 y_ref = 1 / (1 + 25 * x_array ** 2)
134 delta = abs(y_ref - y_array)
135 delta_max = max(delta)
136
137 Array = np.array([x_array, y_ref, y_array, delta]).T
138 print("delta_max =", delta_max)

```

```

139 print(Array)
140
141 x_plot = np.linspace(min(x), max(x), num=10001, endpoint=True)
142 y_plot = Spline(x, y, edge_type, x_plot, edge_0, edge_n)
143 y_ref_plot = 1 / (1 + 25 * x_plot ** 2)
144
145 fig, ax = plt.subplots(figsize=(16, 10), dpi=300)
146 ax.set_xlabel(r"$x$")
147 ax.set_ylabel(r"$y$")
148 ax.scatter(x_array, y_array, color="r", marker="o")
149 ax.plot(x_plot, y_plot, color="b", label="Spline Interpolation")
150 ax.plot(x_plot, y_ref_plot, color="g", linestyle="-.", label=r"$y=1/(1+25x^2)$")
151 ax.legend(loc="upper left")
152 plt.show()

```

内插效果如图：

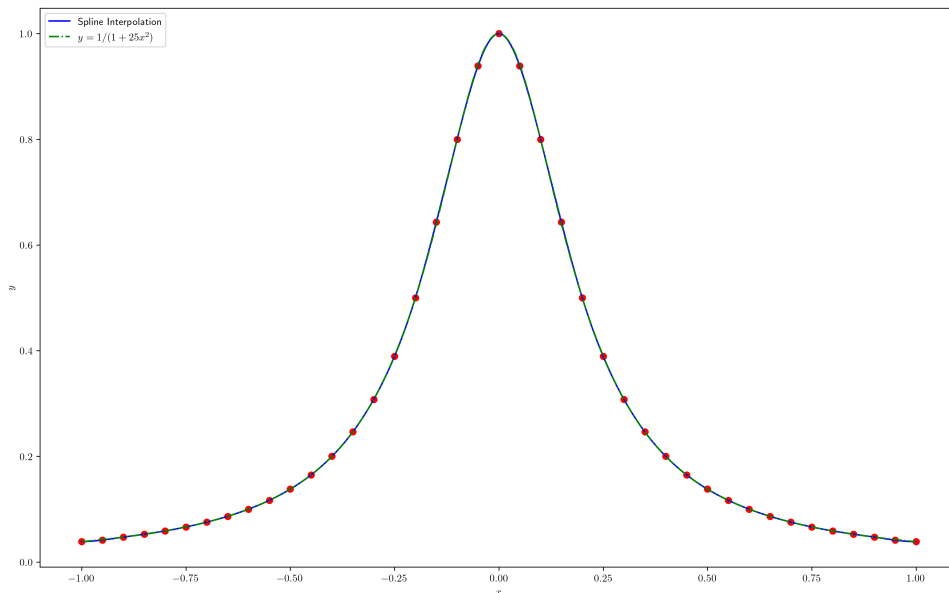


图 4: 使用三次样条函数进行内插

通过计算等距的 10001 个节点 ξ 的误差 $\delta = |f(\xi) - S_d(\xi)|$ 估计最大误差为：

$$\delta_{max} = 0.0031689146$$

每个节点和区间中点的 x , $f(x)$, $S_d(x)$ 以及 $|f(x) - S_d(x)|$ 列表如下：

x	$f(x)$	$S_d(x)$	$ f(x) - S_d(x) $
-1.00	3.84615385e-02	3.84615385e-02	0.00000000e+00
-0.95	4.24403183e-02	4.16088317e-02	8.31486638e-04

x	$f(x)$	$S_d(x)$	$ f(x) - S_d(x) $
-0.90	4.70588235e-02	4.70588235e-02	2.77555756e-16
-0.85	5.24590164e-02	5.26792435e-02	2.20227074e-04
-0.80	5.88235294e-02	5.88235294e-02	8.32667268e-17
-0.75	6.63900415e-02	6.63276137e-02	6.24277479e-05
-0.70	7.54716981e-02	7.54716981e-02	9.71445147e-17
-0.65	8.64864865e-02	8.64914336e-02	4.94713838e-06
-0.60	1.00000000e-01	1.00000000e-01	2.49800181e-16
-0.55	1.16788321e-01	1.16782104e-01	6.21685096e-06
-0.50	1.37931034e-01	1.37931034e-01	1.94289029e-16
-0.45	1.64948454e-01	1.64865836e-01	8.26180987e-05
-0.40	2.00000000e-01	2.00000000e-01	1.11022302e-16
-0.35	2.46153846e-01	2.46267816e-01	1.13970091e-04
-0.30	3.07692308e-01	3.07692308e-01	2.77555756e-16
-0.25	3.90243902e-01	3.89419663e-01	8.24239000e-04
-0.20	5.00000000e-01	5.00000000e-01	1.66533454e-16
-0.15	6.40000000e-01	6.43168915e-01	3.16891462e-03
-0.10	8.00000000e-01	8.00000000e-01	0.00000000e+00
-0.05	9.41176471e-01	9.38866217e-01	2.31025403e-03
0.00	1.00000000e+00	1.00000000e+00	0.00000000e+00
0.05	9.41176471e-01	9.38866219e-01	2.31025143e-03
0.10	8.00000000e-01	8.00000000e-01	2.22044605e-16
0.15	6.40000000e-01	6.43168907e-01	3.16890683e-03
0.20	5.00000000e-01	5.00000000e-01	1.11022302e-16
0.25	3.90243902e-01	3.89419692e-01	8.24210434e-04
0.30	3.07692308e-01	3.07692308e-01	2.22044605e-16
0.35	2.46153846e-01	2.46267710e-01	1.13863617e-04
0.40	2.00000000e-01	2.00000000e-01	5.55111512e-16
0.45	1.64948454e-01	1.64866233e-01	8.22207703e-05
0.50	1.37931034e-01	1.37931034e-01	1.11022302e-16
0.55	1.16788321e-01	1.16780621e-01	7.69969104e-06
0.60	1.00000000e-01	1.00000000e-01	4.44089210e-16
0.65	8.64864865e-02	8.64969677e-02	1.04811703e-05
0.70	7.54716981e-02	7.54716981e-02	2.77555756e-17
0.75	6.63900415e-02	6.63069605e-02	8.30810353e-05
0.80	5.88235294e-02	5.88235294e-02	1.87350135e-16
0.85	5.24590164e-02	5.27563226e-02	2.97306191e-04
0.90	4.70588235e-02	4.70588235e-02	5.96744876e-16
0.95	4.24403183e-02	4.13211685e-02	1.11914982e-03

x	$f(x)$	$S_d(x)$	$ f(x) - S_d(x) $
1.00	3.84615385e-02	3.84615385e-02	2.80331314e-15

与 Neville 算法和 Chebyshev 算法比较可见三次样条函数在变化剧烈的 $x = 0$ 附近和边界 $|x| = 1$ 附近都有较好的内插效果，误差显著减小。

3.(a) $x(t)$ 与 $y(t)$ 的精确数值列表如下:

t	x	y
0.00000000e+00	0.00000000e+00	0.00000000e+00
3.92699082e-01	7.03261419e-02	2.91300418e-02
7.85398163e-01	2.07106781e-01	2.07106781e-01
1.17809725e+00	2.36236823e-01	5.70326142e-01
1.57079633e+00	0.00000000e+00	1.00000000e+00
1.96349541e+00	-5.29130042e-01	1.27743292e+00
2.35619449e+00	-1.20710678e+00	1.20710678e+00
2.74889357e+00	-1.77743292e+00	7.36236823e-01
3.14159265e+00	-2.00000000e+00	2.44929360e-16

3.(b) 过这些点的三次样条函数如下:

$S_d(X; t_i) = s_0 + s_1 t^1 + s_2 t^2 + s_3 t^3$			
s_0	s_1	s_2	s_3
5.00000000e-01	3.12709949e-01	0.00000000	-7.94575394e-02
6.83243333e-01	-3.87228023e-01	8.91188705	-4.57689773e-01
-3.80796031e+00	8.19033909e+00	-4.56946012	7.01095898e-01
-2.12428116e+00	6.04661222e+00	-3.65963418	5.72381857e-01
3.33705793e+01	-2.78484827e+01	7.12950964	-5.72381857e-01
4.11653901e+01	-3.38032796e+01	8.64588621	-7.01095898e-01
-8.00971081e+01	4.33948244e+01	-7.73606027	4.57689773e-01
-1.72446451e+01	9.09786382e+00	-1.49773933	7.94575394e-02

$S_d(Y; t_i) = s_0 + s_1 t^1 + s_2 t^2 + s_3 t^3$			
s_0	s_1	s_2	s_3
0.00000000e+00	4.27956500e-02	0.00000000	3.58111040e-01
4.55184500e-01	-1.69588107e+00	2.21375195	-5.81434469e-01
-5.18450015e-02	-7.27526050e-01	1.59727800	-4.50614633e-01
-1.47618635e+01	1.80018512e+01	-6.35171659	6.73938061e-01
-1.47618635e+01	1.80018512e+01	-6.35171659	6.73938061e-01
5.33400740e+01	-3.40241967e+01	6.89660773	-4.50614633e-01
6.70298705e+01	-4.27393919e+01	8.74602958	-5.81434469e-01
-8.90984130e+01	4.24557673e+01	-6.75023408	3.58111040e-01

内插效果如图：

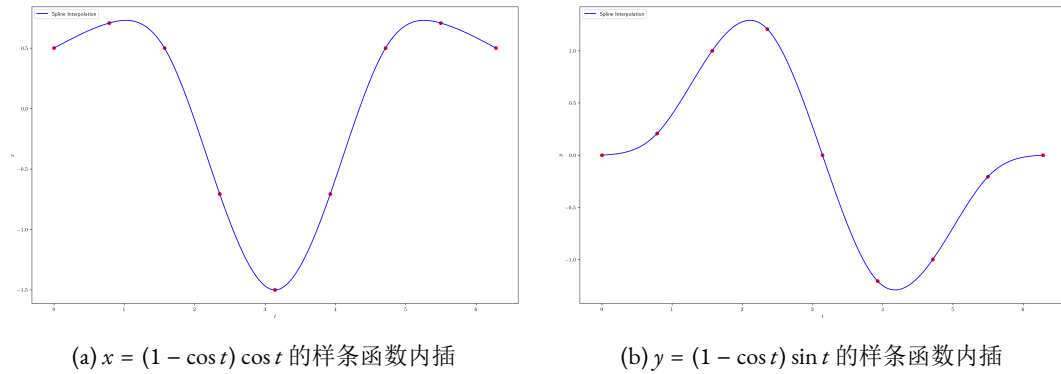


图 5: 分别对 $x-t$, $y-t$ 进行样条函数内插

这里使用的边界条件是: $S''_d(X;0) = S''_d(X;2\pi) = 0$, $S''_d(Y;0) = S''_d(Y;2\pi) = 0$

3.(c) 参数形式的曲线如下图:

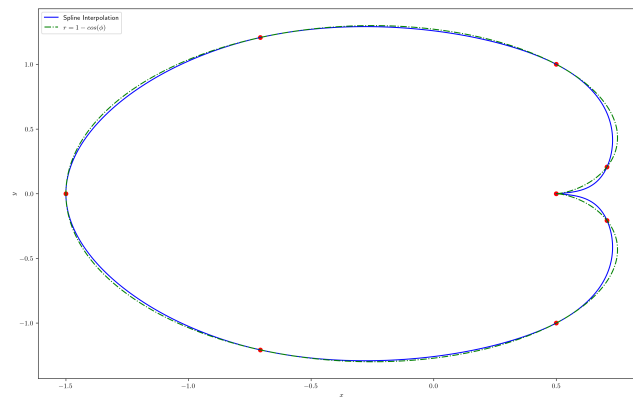


图 6: 参数形式的心形线

不同边界条件给出的参数形式曲线如下图所示, 可见 Neumann 边界条件和周期性边界条件的效果都不如直接给定端点处二阶导 $x''(t) = y''(t) = 0$ 的效果:

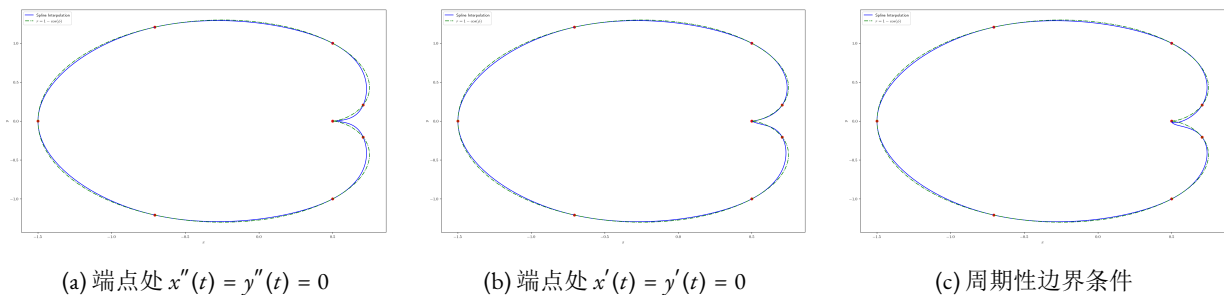


图 7: 不同边界条件给出的参数形式曲线

3.(d) 因为三次样条函数是使得函数在区间 $[a, b]$ 上的模:

$$\|f\| = \int_a^b |f''(x)|^2 dx$$

最小的函数，而这个模可以视为对 $f(x)$ 的曲率 $\rho(x) \approx |f''(x)|$ 的近似。三次样条函数是 $f(x)$ 最小曲率的近似，是“最光滑”的内插函数。它满足在每一个点左右有相同的函数值、一阶导数和二阶导数值，即函数连续、斜率和曲率相同，可以平滑连接所有点。

4.(a) 利用梯形法计算积分，代码如下：

```
1  import numpy as np
2
3
4  def Trapezoidal(func, a, b, N): # \int_a^b func(x) dx
5      if a != b:
6          x_array = np.linspace(a, b, N + 1)
7      elif a == b:
8          return 0
9      dx = (b - a) / N
10     y_array = func(x_array)
11     I = dx * (sum(y_array) - 0.5 * y_array[0] - 0.5 * y_array[-1])
12     return I
13
14
15 def func(x):
16     return np.exp(-(x ** 2)) * np.cos(x)
17
18
19 Array_a = np.zeros((18, 2))
20 for a in range(1, 19):
21     I = Trapezoidal(func, -a, a, a * 20)
22     Array_a[a - 1, 0] = I
23     Array_a[a - 1, 1] = abs(np.exp(-0.25) * np.pi ** 0.5 - I)
24 print(Array_a)
25
26 Array_N = np.zeros((18, 2))
27 for N in range(1, 19):
28     I = Trapezoidal(func, -10, 10, 2 ** N)
29     Array_N[N - 1, 0] = I
30     Array_N[N - 1, 1] = abs(np.exp(-0.25) * np.pi ** 0.5 - I)
31 print(Array_N)
```

固定积分步长为 0.1，选取不同的积分区间，得到的积分值 I 和误差 $\delta = \left| \sqrt{\pi} e^{\frac{1}{4}} - I \right|$ 列表如下：

$[-a, a]$	I	δ	$[-a, a]$	I	δ
$[-1, 1]$	1.31117012	6.92183287e-02	$[-10, 10]$	1.38038845	6.66133815e-16
$[-2, 2]$	1.38524609	4.85764216e-03	$[-11, 11]$	1.38038845	4.44089210e-16
$[-3, 3]$	1.38042838	3.99288253e-05	$[-12, 12]$	1.38038845	4.44089210e-16
$[-4, 4]$	1.38038846	1.63180258e-08	$[-13, 13]$	1.38038845	4.44089210e-16
$[-5, 5]$	1.38038845	1.05937481e-12	$[-14, 14]$	1.38038845	4.44089210e-16
$[-6, 6]$	1.38038845	2.22044605e-16	$[-15, 15]$	1.38038845	2.22044605e-16
$[-7, 7]$	1.38038845	2.22044605e-16	$[-16, 16]$	1.38038845	2.22044605e-16
$[-8, 8]$	1.38038845	4.44089210e-16	$[-17, 17]$	1.38038845	2.22044605e-16
$[-9, 9]$	1.38038845	8.88178420e-16	$[-18, 18]$	1.38038845	4.44089210e-16

可见随着积分区间的增大，误差整体呈现减小的趋势，最后逼近机器精度的极限，误差会有波动。

固定积分区间为 $[-10, 10]$ ，选取不同的积分步长 $\Delta x = \frac{20}{2^N}$ ，得到的积分值 I 和误差 δ 列表如下：

N	I	δ	N	I	δ
1	10.0000000	8.61961155e+00	10	1.38038845	1.55431223e-15
2	5.00000000	3.61961155e+00	11	1.38038845	4.44089210e-16
3	2.49226714	1.11187870e+00	12	1.38038845	1.77635684e-15
4	1.41136986	3.09814175e-02	13	1.38038845	1.33226763e-15
5	1.38038845	2.23901231e-09	14	1.38038845	1.24344979e-14
6	1.38038845	4.44089210e-16	15	1.38038845	2.39808173e-14
7	1.38038845	6.66133815e-16	16	1.38038845	1.35447209e-14
8	1.38038845	4.44089210e-16	17	1.38038845	5.70654635e-14
9	1.38038845	4.44089210e-16	18	1.38038845	2.23376873e-13

可见随着积分区间的增大，误差先减小后增大，区间增大时，函数值相比于机器误差不可忽略，导致总体误差增大。

利用外推积分法计算积分，代码如下：

```

1 import numpy as np
2
3
4 def Extrapolation(func, a, b, m):
5     h = [abs(b - a) / (2 ** i) for i in range(m + 1)]
6     Nev_array = np.zeros((m + 1, m + 1))
7     for i in range(m + 1):
8         if a != b:
9             x_array = np.linspace(a, b, 2 ** i + 1, endpoint=True)
10            elif a == b:
11                return 0
12            dx = (b - a) / 2 ** i

```

```

13     y_array = func(x_array)
14     I = dx * (sum(y_array) - 0.5 * y_array[0] - 0.5 * y_array[-1])
15     Nev_array[i, 0] = I
16     for i in range(1, m + 1):
17         for k in range(1, i + 1):
18             Nev_array[i, k] = Nev_array[i, k - 1] + (
19                 Nev_array[i, k - 1] - Nev_array[i - 1, k - 1]
20                 ) / ((h[i - k] / h[i]) ** 2 - 1)
21     return Nev_array[m, m]
22
23
24 def func(x):
25     return np.exp(-(x ** 2)) * np.cos(x)
26
27
28 Array_a = np.zeros((18, 2))
29 for a in range(1, 19):
30     I = Extrapolation(func, -a, a, 10)
31     Array_a[a - 1, 0] = I
32     Array_a[a - 1, 1] = abs(np.exp(-0.25) * np.pi ** 0.5 - I)
33 print(Array_a)
34
35 Array_m = np.zeros((18, 2))
36 for m in range(1, 19):
37     I = Extrapolation(func, -10, 10, m)
38     Array_m[m - 1, 0] = I
39     Array_m[m - 1, 1] = abs(np.exp(-0.25) * np.pi ** 0.5 - I)
40 print(Array_m)

```

选取不同的积分区间，得到的积分值 I 和误差 $\delta = \left| \sqrt{\pi} e^{\frac{1}{4}} - I \right|$ 列表如下：

$[-a, a]$	I	δ	$[-a, a]$	I	δ
$[-1, 1]$	1.31234873	6.80397216e-02	$[-10, 10]$	1.38038845	1.26565425e-14
$[-2, 2]$	1.38522290	4.83445621e-03	$[-11, 11]$	1.38038845	3.70814490e-14
$[-3, 3]$	1.38042719	3.87411007e-05	$[-12, 12]$	1.38038845	6.12843110e-14
$[-4, 4]$	1.38038846	1.52084974e-08	$[-13, 13]$	1.38038845	6.94999613e-14
$[-5, 5]$	1.38038845	1.01474384e-12	$[-14, 14]$	1.38038845	8.65973959e-14
$[-6, 6]$	1.38038845	6.66133815e-16	$[-15, 15]$	1.38038845	8.05355782e-13
$[-7, 7]$	1.38038845	6.66133815e-16	$[-16, 16]$	1.38038845	2.96984659e-12
$[-8, 8]$	1.38038845	2.66453526e-15	$[-17, 17]$	1.38038845	7.99982303e-12
$[-9, 9]$	1.38038845	2.88657986e-15	$[-18, 18]$	1.38038845	1.78181914e-11

可见随着积分步长的减小，误差先减小后增大，步长过小时，机器误差不可忽略且误差累计次数增多，导致总体误差增大。

选取不同的外推阶数 m ，选取积分步长为：

$$\Delta x = \frac{20}{2^i}, i = 0, 1, \dots, m$$

得到的积分值 I 和误差 δ 列表如下：

m	I	δ	m	I	δ
1	13.3333333	1.19529449e+01	10	1.38038845	1.26565425e-14
2	2.66666667	1.28627822e+00	11	1.38038845	2.22044605e-16
3	1.52674646	1.46358017e-01	12	1.38038845	2.44249065e-15
4	1.00018790	3.80200547e-01	13	1.38038845	1.11022302e-15
5	1.39930803	1.89195831e-02	14	1.38038845	1.75415238e-14
6	1.38082752	4.39067991e-04	15	1.38038845	2.86437540e-14
7	1.38037481	1.36356818e-05	16	1.38038845	8.65973959e-15
8	1.38038851	5.99889054e-08	17	1.38038845	7.68274333e-14
9	1.38038845	6.02289330e-11	18	1.38038845	2.97317726e-13

可见随着外推阶数的增大，误差先减小后增大，阶数过大、步长过小时，机器误差不可忽略且误差累计次数增多，导致总体误差增大。

4.(b) 该积分应该采用 Gauss-Hermite 积分公式，积分代码如下：

```

1  import numpy as np
2
3
4  def Gauss_Int(func, poly_type, deg):
5      if poly_type == "chebyshev":
6          x = np.polynomial.chebyshev.chebgauss(deg)[0]
7          w = np.polynomial.chebyshev.chebgauss(deg)[1]
8      elif poly_type == "hermite":
9          x = np.polynomial.hermite.hermgauss(deg)[0]
10         w = np.polynomial.hermite.hermgauss(deg)[1]
11     elif poly_type == "laguerre":
12         x = np.polynomial.laguerre.laggauss(deg)[0]
13         w = np.polynomial.laguerre.laggauss(deg)[1]
14     elif poly_type == "legendre":
15         x = np.polynomial.legendre.leggauss(deg)[0]
16         w = np.polynomial.legendre.leggauss(deg)[1]
17     I = sum(func(x) * w)
18     return I
19
20
21 def func(x):
22     return np.cos(x)
23
24
25 for deg in [5, 10, 15, 20]:
26     I = Gauss_Int(func, "hermite", deg)
27     print(I)
28     print(np.exp(-0.25) * np.pi ** 0.5 - I, "\n")

```

其中积分节点和权重系数由函数 `numpy.polynomial.chebyshev.chebgauss(deg)` 给出。

选取阶数 deg 分别为 5、10、15、20，积分结果 I 和误差 $\delta = \left| \sqrt{\pi} e^{\frac{1}{4}} - I \right|$ 分别为：

deg	I	δ
5	1.3803900759356564	-1.6288925135388155e-06
10	1.3803884470431410	1.9984014443252818e-15
15	1.3803884470431430	0.0
20	1.3803884470431427	2.220446049250313e-16

可见在阶数 $deg > 10$ 之后积分结果趋于稳定，误差仅仅受机器精度限制。